TMKMAKE

New packaging for QUSRTOOL/TMKMAKE. Library QUSRTOOL is part of the IBM product 5770SS1 opt. 7. This document is based on the information in the member TMKINFO into the file QATTINFO.

The TMKMAKE command allows you to:

- maintain relationships between parts such as source files and program objects
- maintain process procedures such as printing files and transferring files.

This document describes:

- how to install the TMKMAKE command
- which TMKMAKE source files are shipped
- a list of objects created by the installer
- how to delete the TMKMAKE command
- the TMKMAKE command syntax
- a detailed description of the tool
- how to use the tool

SUMMARY

- TMKMAKE
  - INTRODUCTION
  - INSTALLATION
    - To delete the TMKMAKE command
  - GLOSSARY
  - COMMAND SYNTAX
    - TARGET
    - SRCFILE
    - SRCMBR
    - MACRO
    - OPTION
    - MSGLOG
    - MARGINS
    - RTNCDE
    - SRVOPT
  - SYNTAX
    - Comments
    - Continuation Line
    - Escape Character
  - DESCRIPTION FILE RULES
    - Command Prefixes
    - Rule Command Execution
    - Macro Definitions
    - Built-in Macros

    - Extended Built-in Macros
    - Dynamic Dependency
    - Output Translation
    - Target Keywords
    - Object Specification Syntax
    - Object Specification Examples
    - Include Directives
    - Conditional Directives
    - Suffixes and Transformation Rules
    - Inference Rule Specification
    - Inference Rule Examples
    - Built-in Inference Rules
  - Link Editor Libraries
  - ILE Enablement
  - EXAMPLES
    - Example 1
    - Example 2
  - Recursive TMKMAKE Command
  - Messages
  - BIBLIOGRAPHY

INTRODUCTION

The TMKMAKE command is an implementation of the MAKE tool which maintains up-to-date versions of programs consisting of multiple parts. The TMKMAKE command reads a user-supplied description file, which describes the rules, part dependencies and commands to be executed for the parts being maintained. The TMKMAKE command verifies that all targets specified are up-to-date. If one or more of the dependants have been modified since the target was last updated, the commands associated with the target rule are invoked. A target file which is not found is considered out-of-date.

The TMKMAKE command supports the following features: macro definition and substitution, include and conditional directive processing, built-in rules definition, suffix transformation, Extended Program Model (EPM) LIBFILE and System C/400 TXTLIB library maintenance, Integrated Language Environment (ILE) programming model, ILE binding directories, and recursive processing.

INSTALLATION

To use the TMKMAKE command, you must install it. This is done by calling the *PASERIE/INSTALL* utility:

```
PASERIE/INSTALL GIT_USER(AndreaRibuoli) PACKAGEN(TMKMAKE)
```

The **source** for everything you need has been provided in the GitHub repository

FILE	MEMBER	DESCRIPTION

FILE	MEMBER	DESCRIPTION
QCMDSRC	TMKMAKE	Command source for TMKMAKE command
QCLSRC	TMKTCHMB	TMKMAKE CL source
QCLSRC	TMKTCHOB	TMKMAKE CL source
QCSRC	TMKUTIL	C program source
QCSRC	TMKPARSE	C program source
QCSRC	TMKOPTIO	C program source
QCSRC	TMKMSGHD	C program source
QCSRC	TMKMAKER	C program source
QCSRC	TMKMAKEC	C program source
QCSRC	TMKFILE	C program source
QCSRC	TMKDICT	C program source
QCSRC	TMKBUILT	C program source
QCSRC	TMKEXPR	C program source
H	TMKUTIL	C header file
H	TMKPARSW	C header file
H	TMKOPTIO	C header file
H	TMKOPNA	C header file
H	TMKMSGHD	C header file
H	TMKMAKER	C header file
H	TMKFILE	C header file
H	TMKEXPR	C header file
H	TMKDICT	C header file
H	TMKBUILT	C header file
H	TMKBASE	C header file
QMAKSRC	BUILTIN	TMKMAKE file

The following **objects** are created:

OBJECT NAME	OBJECT TYPE	DESCRIPTION
TMKMAKEC	*PGM	MAKE Program tool
TMKTCHMB	*PGM	MAKE CL utility

OBJECT NAME	OBJECT TYPE	DESCRIPTION
TMKTCHOB	*PGM	MAKE CL utility
TMKMSGF	*MSGF	MAKE TOOL MESSAGES
QMAKSRC	*FILE	MAKE Tool Built-in
TMKMAKE	*CMD	MAKE command

Note: The QMAKSRC file will be created if it does not already exist.

To delete the TMKMAKE command

To delete the TMKMAKE tool delete the TMKMAKE library:

```
DLTLIB LIB(TMKMAKE)
```

GLOSSARY

- **make file**: the description file is a source member usually stored in source file QMAKSRC. It contains the directives for install or update the software package.
- **macro**: macros are just like the variables
- **built-in file**: it contains the inference rules. It is stored in `BUILTIN` member of source file `QMAKSRC` into the library `TMKMAKE`.

COMMAND SYNTAX

The TMKMAKE command helps maintain up-to-date versions of programs consisting of multiple parts. The command only updates the parts that have changed since the last time the TMKMAKE command was invoked.

TARGET

Specifies the name of the target to process. The target name is folded to uppercase before it is processed unless it begins and ends with a single quotation (') mark. A maximum of 30 targets can be specified. The possible target names are:

*FIRST (default)	Specifies the first target listed in the description file.
target_name	Specifies a user-defined target.

SRCFILE

Specifies the name of the description file to process. It consists of a library name and a file name. The possible library names are:

SRCFILE library	
*LIBL (default)	Searches the library list for the library containing the description file.
*CURLIB	Searches the current library for the description file.
library_name	Searches a user-defined library for the description file.

The possible file names are:

SRCFILE filename	
QMAKSRC (default)	Specifies QMAKSRC as the source file.
file_name	Specifies a user-defined file.

SRCMBR

Specifies the member in the description file to process.

*FIRST (default)	Specifies the oldest member created in the source file.
*ALL	Processes all the members created in the source file alphabetically.
member_name	Specifies a user-defined member.

MACRO

User macro(s) can be defined on the command line in the form of `macro=text`. A macro defined on the command line overrides a macro of the same name in the description file. The TMKMAKE command supports up to 50 user-defined macros.

*NONE (default)	Specifies that no user macros are used.
'macro_name=replacement_text'	Specifies a user-defined macro. The TMKMAKE command supports up to 3000 characters of replacement text. The macro name and replacement text are case sensitive.

OPTION

Specifies the options to use when the TMKMAKE command is invoked. You can specify them in any order. Separate them with a blank space. If you specify an option group more than once, the last option specified is used. The possible options are:

*NOIGNORE *(default)*	Does not ignore exceptions or return codes generated by the invoked commands defined in a target rule.
*IGNORE	Ignore exceptions or return codes generated by the invoked commands defined in a target rule. This is equivalent to specifying the *.IGNORE* keyword or the `-` command prefix in the description file.

*NOSILENT *(default)*	Outputs the command text to the output device before it is invoked.
*SILENT	Does not output the command text to the output device before it is invoked. This is equivalent to specifying the *.SILENT* keyword or the @ command prefix in the description file.
*BIRULES *(default)*	Uses a set of built-in rules in the BUILTIN.QMAKSRC file as well as user-defined rules in the description file.
*NOBIRULES	Does not use the default set of built-in rules.
*EXEC *(default)*	Executes commands defined in the target rule if a dependent is out-of-date.
*NOEXEC	Does not execute commands defined in the target rules.
*NOTOUCH *(default)*	Does not explicitly change the date of a target and the date of all dependents associated with that target.
*TOUCH	Explicitly changes the date of a target and the date of all dependents associated with that target.
*NODEBUG *(default)*	Does not display debug information.
*DEBUG	Displays debug information to the session manager. The trace includes information such as macro definitions, and target and dependent part dates used by the TMKMAKE command.

MSGLOG

Redirects all the TMKMAKE command messages and the user messages defined in the make description file to the output device. The output device can be one of the following:

*JOBLOG *(default)*	Directs messages to the job log.
*SESSION	Directs messages to the session manager.

MARGINS

Specifies the left and right margins of the text in the description file which the TMKMAKE command processes. If these values are specified incorrectly, a warning message is issued and the default margin values of the description file are used.

*SRCFILE *(default)*	Uses the margin values of the description file. If the description file is a physical file, the left margin is 1 and the right margin is the record length. If the source file is a source physical file, the left margin is 13 and the right margin is the record length.

left_margin, right_margin	Specifies the values for the left and right margins. These values must be greater than or equal to 1, and less than or equal to 1024, respectively. The left margin value must be less than the right margin value.

RTNCDE

Determines if a command executed successfully by examining the severity value for an exception and/or a return code.

*EXCEPTION *(default)*	Checks for any exception greater than the severity level specified.
*LURC	Checks for any return code greater than the severity level specified.
*BOTH	Checks for both the *EXCEPTION and the *LURC conditions.

The possible severity values are:

30 *(default)*	Specifies a severity limit of 30.
severity_value	Specifies a user-defined severity limit.

SRVOPT

Internal service options for debugging.

SYNTAX

Comments

Any line in the description file beginning with a number sign, (#), is a comment line and is ignored by the TMKMAKE command. If the £ appears in the middle of a text line, the £ and everything after it are ignored. A blank line is ignored by the TMKMAKE command.

Continuation Line

A line in the description file can be continued to another line by using a backlash (\) or a plus sign (+) at the end of the line. Continuation has a higher precedence than comments and description file directives on the same line.

Escape Character

No macro substitution is applied to characters in the description file that are preceded by a backslash (\). However, the # character is exempt from this rule because it is used to indicate comments in the description file and is processed before macro substitution takes place. This feature is useful for characters such as & and $ which usually reference macros in the description file.

DESCRIPTION FILE RULES

Description file rules have the following format:

```
target1 (target2...) :(:) (dependent1...) (£ comment text...)
 (commands text (£ comment text...)
 ...
 )
```

Any items enclosed in brackets are optional. The target must start at the first column from the left margin and the commands specified for each rule must be indented by at least one white space character. A sequence of commands can be associated with a target but each command must be defined on a separate line.

Every rule in a description file is composed of the following four parts:

- **Target Definition**: Although a target may appear in more than one dependency rule, all the rules must be of the same rule type. The dependents from all the rules of the target are concatenated and treated as one dependency rule. Only one command sequence definition can be defined for a target. If there is more than one target in the definition, the TMKMAKE command applies the rule types, dependent definitions and command sequence information to each of the targets as if it were defined by an individual rule.

- **Rule Type**: A single colon (`:`) or double colon (`::`) must be specified for every rule. The single colon is used to define parts dependency and the double colon is used to maintain EPM LIBFILE library objects and System C/400 TXTLIB library objects.

- **Dependent Definition**: A list of dependents can be specified in this optional field. If any dependents are newer than the target, the command sequence defined by the rule is executed. If no command sequence exists, the TMKMAKE command searches for a built-in inference rule. If this is not found, the command sequence associated with the .DEFAULT keyword is used. All the dependent definitions must exist, or appear as a target in one of the rules in the description file; otherwise the TMKMAKE command outputs an error message and stops.

- **Command Sequence Definition**: A sequence of commands can be associated with a dependency rule. If any dependents are newer than the target, the associated commands are executed. If no command exists, the TMKMAKE command invokes a built-in inference rule if it exists. A target with multiple rules (`::`) will only execute the commands associated with that rule if the target is out-of-date with the dependents of the rule. If the target us out-of-date for all rules, the commands associated with all the rules are executed. If more than one command is associated with a rule, each command is invoked, but the commands are not invoked in an atomic fashion. If a command execution fails, the state of all the opened files and other job attributes may or may not be consistent with other commands which had been executed before.

Command Prefixes

The following prefixes affect command execution. One or more blanks may separate the prefixes and the commands. No blank spaces should appear between multiple prefixes.

- `@`: Inhibits the output of the command text to the output device before the command that it prefixes is invoked. This is equivalent to specifying the .SILENT keyword in the description file or using the OPTION (*SILENT).
- `-`: Ignores any exception and/or return code generated by the command being invoked. This is equivalent to specifying the .IGNORE keyword in the description file or using the OPTION (*IGNORE).
- `-n`: Ignores exception and/or return code severity values less than or equal to n. If -n is specified in combination with other prefixes, it must be followed by one or more white spaces.

Rule Command Execution

The TMKMAKE command processes the targets specified in the TARGET parameter or it uses the first target in the description file. The TMKMAKE command recursively checks the dependency paths for each target, to ensure that all the targets are up-to-date before the commands associated with each target are executed.

If a target is out-of-date, the associated command sequence is executed. Each command is output to the output device, unless the OPTION (*SILENT), .SILENT keyword, or '@' command prefix is specified. The command text is passed to the system command interpreter.

The TMKMAKE command stops when an exception and/or return code exceeds the severity limit specified, unless the OPTION (*IGNORE), .IGNORE keyword, or '-' command prefix is specified.

If a target is out of date and no explicit or built-in rules exist to build the target, the command sequence associated with the .DEFAULT target keyword is used. If the .DEFAULT target keyword is not defined, the TMKMAKE command outputs a message and stops processing.

If the `echo` command appears in the command sequence of a rule, all the text characters following the echo command, except comments, will be output to the output device exactly as they are entered in the description file.

Macro Definitions

Macro definitions allow text substitution in the description file by associating a symbolic name to a string of characters. A macro definition has the form: `Macro_name = (Replacement_text)`.

All white space before the macro name, before and after the '=', and after replacement text, is ignored. The macro name and replacement text are **case sensitive**.

Macros can be referenced in the description file using the prefix symbols `&` and `\$` with the macro name. These prefixes can be used interchangeably. If no replacement text exists when a macro is defined, it is replaced with no text when the macro is referenced. If a macro is referenced but has not been defined, the macro is replaced with no text. The TMKMAKE command supports up to 3000 characters of replacement text.

The TMKMAKE command resolves nested macro names in the form of `\$(\$(macro_name))`. The innermost macro name is expanded, and its value is taken as another macro name that can also be expanded. Parentheses can be used to isolate ambiguous macro expansions. Thirty-two levels of nested macro expansion are supported.

A macro defined when the TMKMAKE command is invoked overrides a macro of the same name defined in the description file.

Built-in Macros

The following are TMKMAKE command built-in macros.

macro	definition
`$@`	This macro specifies the name of the current target.
`$?`	This macro specifies a list of prerequisites that are newer than the current target. This macro is evaluated only for the explicit rules defined in a description file.
`$<`	This macro is evaluated to the name of the related prerequisite which caused the command execution to take place, if the command is executed from an implicit rule.
`$*`	This macro is evaluated to the prefix shared by the target and the dependent which caused the command execution to take place, if the command is executed from an implicit rule.
`$**`	This macro expands to a list of all dependents.
`$%`	This macro references a LIBFILE and/or TXTLIB member for a '.a' type link editor library defined in a rule. This macro is set when `$@` is evaluated.

If a single `$` symbol is required in the description file, enter `$$` in the description file and the TMKMAKE command will translate the `$$` sequence into a single `$` character. To get the `$@` character sequence you can use `$$\@`.

Extended Built-in Macros

The following are additional built-in macros.

L	M	F	T	S
`$(*L)`	`$(*M)`	`$(*F)`	`$(*T)`	`$(*S)`
`$(<L)`	`$(<M)`	`$(<F)`	`$(<T)`	`$(<S)`
`$(@L)`	`$(@M)`	`$(@F)`	`$(@T)`	`$(@S)`

The meaning of the `$*`, `$<`, and `$@` are the same as the built-in macros with the following extended names.

- **L**: Refers to the Library name portion of the macro, if it is defined.
- **M**: Refers to the Member name of a database file, an EPM LIBFILE member, a System C/400 TXTLIB member, or no substitute text for a non-database object.
- **F**: Refers to the File name of a database file or the object name for a non-database object.
- **T**: Refers to the object Type.
- **S**: Refers to the SEU type of a database file.

Dynamic Dependency

Macro text in a dependency rule can be expanded dynamically and substituted for the current target name. The TMKMAKE command supports the following dynamic dependency macros which can be used only in the dependent definition.

`\$\$@`	The current target to the left of the colon.
`\$\$(@L)`	The Library part of $$@
`\$\$(@M)`	The Member part of $$@
`\$\$(@F)`	The File part of $$@
`\$\$(@T)`	The Type part of $$@
`\$\$(@S)`	The SEU part of $$@

Output Translation

The TMKMAKE command translates and expands all the macros before it interprets the content of the description file. Output translation is a special form of macro translation which allows the macro to replace one string of characters with another string of characters, once, when the macro is referenced.

Output translation has the following form: $(Macro_name(text1=(text2))). The replacement texts are case sensitive. The macros defined in the text will not be expanded more.

Target Keywords

The following keywords can be used as the target definition of a rule in a description file. The keywords must appear in uppercase.

- `.SUFFIXES`: Defines the order of the file name suffixes used by the TMKMAKE command to transform an object with one suffix into an object with another suffix. If the .SUFFIXES target appears more than once in a description file, the dependents specified in the later .SUFFIXES rule are appended to the existing suffix list. There is no suffix transformation if a target has commands associated with it but does not have dependents. A default list of suffixes is available in the default built-in file.
- `.IGNORE`: Ignores exceptions and return codes generated by the commands associated with the dependency rule. This is equivalent to specifying the OPTION (*IGNORE) or a '-' command prefix for every command in the description file.
- `.SILENT`: Does not output the command text to the output device before each command is invoked. This is equivalent to specifying the OPTION (*SILENT) or a '@' command prefix for every command in the description file.
- `.DEFAULT`: Uses the command sequence associated with the .DEFAULT target to build the target if no explicit commands or built-in rules exist. If the .DEFAULT keyword is not defined, the TMKMAKE command outputs a message and stops processing.
- `.PRECIOUS`: The TMKMAKE command recognizes this keyword in the description file for porting and compatibility purposes but no action is taken.

Object Specification Syntax

The following syntax diagram describes how to specify description file objects.

```
--------------------Member.File------------------------------------->
 !         !   !  !              !                            !   !
 !-*LIBL---!-/--  !              --<FILE-------------------->--   !
 ! =====   !      !                 ====  !               !       !
 !-*CURLIB-!      !                      --,seu_src_type--       !
 --Library--      !                                              !
                  !--Object--------------------------------------!
                  !              !-<PGM>---------!               !
                  !              ! =====         !               !
                  !              !-<MODULE>------!               !
                  !              !-<PGMSET>------!               !
                  !              !-<FILE>--------!               !
                  !              !-<CMD>---------!               !
                  !              !-<LIBFILE>-----!               !
                  !              !-<TXTLIB>------!               !
                  !              !-<BNDDIR>------!               !
                  !              !-<USRSPC>------!               !
                  !              !-<DTAARA>------!               !
                  !              --.........------               !
                  !                                              !
                  ---Object(Mbr)----------------------------------
                                 !               !
                                 !-<LIBFILE> ----!
                                 ! =========     !
                                 --<TXTLIB>-------
```

Notes:

1. Valid object names can begin with the characters A-Z, a-z, $, or @ followed by A-Z, a-z, 0-9, $, @, and _ characters.
2. seu_src_type is the SEU type for the members of a *FILE object.
3. FILE must be specified in uppercase.

Object Specification Examples

```
hello.source<FILE,C) - *LIBL/source(hello)
                     - qualified *FILE object type
                     - SEU 'C' source type

*CURLIB/pgmobj<PGM>  - *CURLIB/pgmobj
                     - fully qualified program name

libfile(mbr)         - *LIBL/libfile, LIBFILE entry mbr
                     - default to LIBFILE *FILE object type
```

Include Directives

Include directives read and process the content of the filename before continuing with the current description file. Include directives can be specified as éinclude filename or include filename. A macro can be used in a filename.

Conditional Directives

`!if` *expression*	Specifies that all lines following the directive to the next nested éelse, éelif, or éendif directive should be processed if the expression results in a non-zero value.
`!else`	Specifies that all lines following the directive to the next nested !endif should be processed if the preceding !if, !elif, !ifdef, or !ifndef expression results in a zero.
`!elif` *expression*	Specifies that all lines following the directive to the next nested éelse éelif or éendif directive should be processed if the expression results in a non-zero value.
`!endif`	Specifies the end of an !if, !else, !elif, !ifdef, or !ifndef statement block.
`!ifdef` *macro*	Same as the éif expression except the condition is true only if the macro variable is currently defined. A macro with null string assigned to it is also considered to be defined.
`!ifndef` *macro*	Same as the éifdef expression except the condition is true only if the macro variable is currently not defined.
`!error` *text*	Instructs the TMKMAKE command to stop processing, output text to the output device and exit the MAKE process. Macros are expanded within the text before output except for built-in and extended built-in macros.
`!undef`	Undefines the macro from the dictionary.

All conditional directives must be entered in **lowercase**.

The TMKMAKE command supports ANSI C defined trigraphs in expressions for éif and éelif conditional directives.

All directives must start in the **first column** of the description file.

The TMKMAKE command supports 32 levels of directive nesting.

The following operators are valid for the directive expressions: `ì`, `!`, `-`(unary minus), `+`(unary plus), `*`, `/`, `%`, `+` (integer addition), `-`(integer subtraction), `<<`, `>>`, `<`, `<=`, `>`, `>=`, `==`, `!=`, `&`, circumflex, `!`, , `!!`.

Suffixes and Transformation Rules

The TMKMAKE command uses a set of built-in inference rules to transform an object with one suffix to an object with another suffix. This feature is used to define dependency rules in the description file without explicitly specifying the dependencies and commands for each target rule. A suffix represents the filename of a *FILE object type which is represented as Member.File. The suffix follows the '.' character.

The suffixes list is the dependents definition list for the .SUFFIXES keyword. The TMKMAKE command scans this list from left to right to transform the target object name from an old suffix to a new suffix. You can append new suffixes to the list using the .SUFFIXES keyword in the description file. There is no suffix transformation if a target has commands associated with it but has no dependents.

If the TMKMAKE command needs to process a target which does not have a rule or command sequence associated with the target, it repeatedly replaces the target suffix with each suffix from the suffixes list and searches in the library list for the first object in the specified library, which matches the new object specification. If no library is specified, the library list is searched. If the object exists and a built-in inference rule is found, the command sequence for this built-in inference rule is associated with the target.

Most system objects can be identified by a single name except database files. In database files, you can specify a member in a file. For example, you can specify the STDIO member in file H as STDIO.H. The TMKMAKE command supports null suffix rules to maintain the relationship between a dependent with a suffix and a target with no suffix, where the default target suffix type is .

Inference Rule Specification

The following syntax diagram describes how to specify inference rule objects.

```
----ObjectSpec------------------------------------------------------>
                           !              !
                           ---ObjectSpec--
```

where ObjectSpec is defined as

```
----.--------FileName-----------------------------------------
        !              !                                !
        !              --<FILE-------------------->-----!
        !                ====  !               !       !
        !                      --,seu_src_type--       !
        !                                              !
        !-----------------<PGM>-------------------------!
        !                 =====                        !
        !-----------------<PGMSET>----------------------!
        !-----------------<CMD>-------------------------!
        !-----------------<LIBFILE>---------------------!
        !-----------------<TXTLIB>----------------------!
        !-----------------<BNDDIR>----------------------!
        !-----------------<USRSPC>----------------------!
        !-----------------<DTAARA>----------------------!
        ------------------.........----------------------
```

Note:

1. seu-src_type is the SEU type for the members of a *FILE object.

Inference Rule Examples

```
.qcsrc            - .qcsrc<FILE>.<PGM>
                  - null suffix inference rule
                  - transforms a QCSRC source file to a *PGM object
```

```
.qcsrc.<LIBFILE>  - .qcsrc<FILE>.<LIBFILE>
                  - transforms a QCSRC source file to a *SRVPGM object
                    and builds it into a *BNDDIR object
```

Built-in Inference Rules

The TMKMAKE command reads the contents of the file, BUILTIN.QMAKSRC, which contains all the predefined built-in rules and macro definitions shipped with the command. The TMKMAKE command searches the *CURLIB first for this file and if it is not found, searches the *LIBL for it. You can redefine these built-in rules by replacing this file with a file containing user-supplied defaults. If the built-in rules file cannot be found, the TMKMAKE command outputs a warning message and continues with normal processing. If the OPTION (*NOBIRULES) is specified, the contents of this file will not be processed. BUILTIN.QMAKSRC can be found in a user-defined library.

Link Editor Libraries

The TMKMAKE command can maintain link editor libraries. It is a collection of software components bound into an object from which a link editor can resolve undefined external references declared in an application program. The TMKMAKE command can maintain the EPM Language LIBFILE object and the System C/400 TXTLIB object.

A link editor library and its members can be specified in the description file as follows:

```
projlib<LIBFILE>         # projlib <LIBFILE>
projlib<TXTLIB>          # fully qualified projlib <TXTLIB>

projlib(MEMBER)          # member MEMBER in projlib <LIBFILE>
projlib(MEMBER)<TXTLIB>  # fully qualified member MEMBER for
                         # System C/400 <TXTLIB> object projlib
```

The following shows a description file that can be used to create an ILE C LIBFILE made up of two programs.

```
com_serv<LIBFILE>::    com_serv(service1)
    CRTCPGM PGM(service1) SRCFILE(QCSRC) SRCMBR(service1)
    EXTPGMINF PGM(service1) FILE(com_serv) OPTION(*UPDATE)

com_serv<LIBFILE>::    com_serv(service2)
    CRTCPGM PGM(service2) SRCFILE(QCSRC) SRCMBR(service2)
    EXTPGMINF PGM(service2) FILE(com_serv) OPTION(*UPDATE)
```

Note the double colon (`::`) rule type used to facilitate multiple rules for the same target.

Since the command sequences for both rules are the same, inference rules and special handling can be used to simplify building the link editor library. Therefore, a .QCSRC.a inference rule can be defined to

compile the program. If both members depend on the include file defs.h, the following dependency rule can be used.

```
com_serv(service1):    defs.h
com_serv(service2):    defs.h
```

When this form of rule is used, a macro $% is evaluated to the referenced member name each time $@ is evaluated. If there is no current library member, $% is evaluated to a null character string. The previous description file can be rewritten as the following:

```
.QCSRC.<LIBFILE>:
    CRTCPGM PGM($(@M)) SRCFILE(QCSRC) SRCMBR($(@M))
    EXTPGMINF PGM($%) FILE($(@F)) OPTION(*UPDATE)

com_serv<LIBFILE>:  com_serv(service1) com_serv(service2)
    @echo LIBFILE com_serv updated

com_serv(service1) com_serv(service2):     defs.h
```

ILE Enablement

The following are built-in reference rules for ILE C.

```
.qcsrc.<MODULE>   - .qcsrc<FILE>.<MODULE>
                  - transforms a QCSRC source file to a *MODULE object
                    and builds the associated MODULE

.qcsrc.<BNDDIR>   - .qcsrc<FILE>.<BNDDIR>
                  - transforms a QCSRC source file to a *MODULE object
                    and builds the associated MODULE
```

The following is an example of a description file to create an ILE C module.

```
 prog<MODULE>:

 prog.qcsrc: def.h
```

The following is an example of a description file to create an ILE program created by binding three ILE C programs.

```
 pgm: prog1<MODULE> prog2<MODULE> prog3<MODULE>

 prog1<MODULE> prog2<MODULE>: defs.h
```

```
    prog3<MODULE>: map.h
```

The following is an example of a description file to maintain a binding directory from ILE C source programs.

```
    mylib/bnddir<BNDDIR>: bnddir(service1)<BNDDIR> bnddir(service2)<BNDDIR>

    service1<MODULE> service2<MODULE>:  sev.h
```

EXAMPLES

The following two examples show two different ways of performing the same function.

Example 1

```
# This makefile is used to create MYPGM

#
# The silent command instructs TMKMAKE not to echo the statements
#

.SILENT:

#
# define macros for the commands and options to be used
#

CCOPTS=OUTPUT(*PRINT)
CC=CRTCMOD
CP=CRTPGM
LIB=GLENTEST

#
# define macros which describe the nesting within the includes
#

header1a = header1a.h
header1b = header1b.h
header2a = header2a.h
header1  = header1.h  $(header1a) $(header1b)
header2  = header2.h  $(header1)  $(header2a)
header3  = header3.h

#
# describe the rule to create the *PGM mypgm
#

mypgm<PGM>: part1<MODULE> part2<MODULE>
       $(CP) PGM($(LIB)/mypgm) MODULE($(LIB)/part1 $(LIB)/part2)
```

```
#
# describe the rules to create the *MODULE objects
#

part1<MODULE>: part1.QCSRC<FILE> $(header1) $(header3)
       $(CC) MODULE($(LIB)/part1) SRCFILE($(LIB)/QCSRC) $(CCOPTS)

part2<MODULE>: part2.QCSRC<FILE> $(header2) $(header3)
       $(CC) MODULE($(LIB)/part2) SRCFILE($(LIB)/QCSRC) $(CCOPTS)
```

Example 2

```
#
# This makefile is also used to create MYPGM
#

# The silent command instructs TMKMAKE not to echo the statements

.SILENT:

#
# Define the hierarchical order of objects to search
#

.SUFFIXES: .<PGM> .<MODULE> .QCSRC<FILE>

#
# define macros for the commands and options to be used
#

CCOPTS=OUTPUT(*PRINT)
CC=CRTCMOD
CP=CRTPGM
LIB=GLENTEST

#
# define macros which describe the nesting within the includes
#

header1a = header1a.h
header1b = header1b.h
header2a = header2a.h
header1  = header1.h  $(header1a) $(header1b)
header2  = header2.h  $(header1)  $(header2a)
header3  = header3.h

#
# describe the rule to create the *PGM mypgm
#
```

```
mypgm<PGM>: part1<MODULE> part2<MODULE>
       $(CP) PGM($(LIB)/mypgm) MODULE($(LIB)/part1 $(LIB)/part2)

#
# describe the rules to create the *MODULE objects
#

part1<MODULE>: $(header1) $(header3)
part2<MODULE>: $(header2) $(header3)

.QCSRC.<MODULE>:
       $(CC) MODULE($(LIB)/$*) SRCFILE($(LIB)/QCSRC) $(CCOPTS)
```

Recursive TMKMAKE Command

The TMKMAKE command can be invoked recursively if it is defined in the description file. This feature is useful when a hierarchy of description files define a set of software subsystems. Normally, if the OPTION (*NOEXEC) is specified, no commands are executed. However, if the OPTION (*NOEXEC) is specified and the character sequence defined by a MAKE macro appears in the command text, a recursive call is executed.

Messages

All user and TMKMAKE command messages are directed to an output device. The default output device is the job log (*JOBLOG). If the OPTION (*SESSION) is specified, all user messages only are directed to the session manager. If the OPTION (*DEBUG) is specified, debug, macro and internal built-in rule messages are directed to the session manager.

The OVRDBF command can be used to capture any information that is sent to the session manager, and put it into a database file.

BIBLIOGRAPHY

- Programming with Make on the AS/400, Part 1, MC Press Online, 31-aug-1998, https://www.mcpressonline.com/programming-other/general/programming-with-make-on-the-as400-part-i
- Programming with Make on the AS/400, Part 2, MC Press Online, 30-sep-1998, https://www.mcpressonline.com/programming-other/general/programming-with-make-on-the-as400-part-2
- Setting up TMKMAKE, in Try AS/400 Wiki, https://try-as400.pocnet.net/wiki/Setting_up_TMKMAKE#Bugs_and_oddities